UNIDAD 6

Sintaxis y clasificación de sentencias SQL

Investigación técnica · Tarea 6.1

Estudiante	Kevin Anselmo Orozco Aguayo
Docente	Rubén Riojas
Materia	Bases de Datos
Motor de base de datos	MySQL 8.0
Fecha	Noviembre de 2026

Documento académico elaborado para la Unidad 6

TAREA 6.1 — SINTAXIS Y CLASIFICACIÓN DE SENTENCIAS SQL

Título	Tarea 6.1 - Sintaxis y Clasificación de Sentencias SQL
Nombre	Kevin Anselmo Orozco Aguayo
Docente	Rubén Riojas
Materia	Bases de Datos
Fecha	Noviembre 2026

ÍNDICE

1. Introducción al Lenguaje SQL
2. Características Generales de SQL
3. Lenguaje de Definición de Datos (LDD / DDL)
4. Lenguaje de Manipulación de Datos (LMD / DML)
5. Diferencias entre LDD y LMD
6. Comandos DDL (LDD)6.1 CREATE6.2 ALTER6.3 DROP6.4 TRUNCATE
7. Comandos DML (LMD)7.1 INSERT7.2 UPDATE7.3 DELETE7.4 SELECT
8. Tabla Taxonómica de Comandos SQL
9. Diferencias entre DDL, DML y DQL
10. Conclusiones

• Bibliografía

1. Introducción al Lenguaje SQL

Historia y Origen

El Lenguaje de Consulta Estructurado, conocido universalmente por sus siglas en inglés SQL (Structured Query Language), tiene sus raíces en los trabajos teóricos realizados por el científico informático Edgar Frank Codd en la década de 1970. En 1970, Codd publicó su artículo seminal "A Relational Model of Data for Large Shared Data Banks" en la revista Communications of the ACM, en el que propuso el modelo relacional como fundamento matemático para la organización y manipulación de datos. Inspirados en este modelo, investigadores de los laboratorios de IBM en San José, California —entre ellos Donald D. Chamberlin y Raymond F. Boyce— desarrollaron entre 1973 y 1974 un lenguaje experimental denominado inicialmente SEQUEL (Structured English Query Language), diseñado para operar sobre el sistema de gestión de bases de datos relacionales experimental de IBM llamado System R. Debido a conflictos de marca registrada con la empresa aeronáutica HAWKER SIDDELEY, el nombre fue abreviado a SQL.

A finales de la década de 1970 y principios de la de 1980, el lenguaje SQL comenzó a ganar terreno en el mercado comercial. Oracle Corporation (entonces llamada Relational Software Inc.) lanzó en 1979 el primer sistema de gestión de bases de datos relacionales comercial basado en SQL, seguido por IBM con sus productos SQL/DS y DB2 en 1981 y 1983 respectivamente. Durante este período, múltiples fabricantes adoptaron variantes propias del lenguaje, lo que generó cierta fragmentación en su implementación. Sin

embargo, la comunidad académica y la industria reconocieron la necesidad de unificar la sintaxis y semántica del lenguaje para garantizar la interoperabilidad entre sistemas. Esta necesidad fue la que impulsó el proceso de estandarización formal que se describirá a continuación.

La primera versión oficial del estándar fue publicada en 1986 por el American National Standards Institute (ANSI) bajo la denominación SQL-86, adoptada al año siguiente por la International Organization for Standardization (ISO) como ISO 9075:1987. Desde entonces, el estándar ha evolucionado de manera continua a través de múltiples revisiones: SQL-89, SQL-92 (también conocido como SQL2, que introdujo mejoras sustanciales en la completitud del lenguaje), SQL:1999 (SQL3, que incorporó elementos de orientación a objetos y expresiones regulares), SQL:2003 (que añadió soporte a XML y funciones de ventana), SQL:2008, SQL:2011, SQL:2016, y finalmente SQL:2023. Cada revisión ha ampliado las capacidades del lenguaje manteniendo compatibilidad hacia atrás en la medida de lo posible.

Importancia en el Desarrollo de Software

SQL ocupa un lugar central en el ecosistema del desarrollo de software moderno por múltiples razones. En primer lugar, prácticamente toda aplicación de software —desde pequeñas aplicaciones móviles hasta sistemas empresariales de misión crítica— requiere alguna forma de persistencia y recuperación de datos. Las bases de datos relacionales, que utilizan SQL como interfaz principal, representan históricamente la tecnología de almacenamiento más ampliamente desplegada en la industria. Según el índice DB-Engines, sistemas como MySQL, PostgreSQL, Microsoft SQL Server y Oracle Database han ocupado consistentemente los primeros puestos en popularidad durante décadas.

Desde una perspectiva de ingeniería de software, SQL permite a los desarrolladores expresar de forma declarativa qué datos necesitan sin especificar el procedimiento exacto para obtenerlos, dejando al motor de base de datos la responsabilidad de optimizar la estrategia de acceso. Esta separación entre la lógica de negocio y la lógica de acceso a datos es un principio fundamental en arquitecturas bien diseñadas y facilita la mantenibilidad a largo plazo del código. Adicionalmente, el conocimiento de SQL es uno de los requisitos técnicos más demandados en ofertas de empleo para desarrolladores de software, ingenieros de datos, analistas de negocio y administradores de bases de datos.

SQL como Estándar ANSI/ISO

El carácter estandarizado de SQL es uno de sus activos más valiosos. Al estar definido por organismos internacionales de normalización (ANSI e ISO), el lenguaje ofrece una base común que permite a los profesionales migrar conocimientos entre diferentes plataformas. No obstante, es importante reconocer que en la práctica cada Sistema Gestor de Bases de Datos Relacionales (SGBDR o RDBMS por sus siglas en inglés) implementa el estándar de forma parcial e incorpora extensiones propias. Por ejemplo, MySQL utiliza la cláusula `LIMIT` para limitar resultados, mientras que SQL Server emplea `TOP` y Oracle hace uso de `ROWNUM` o la cláusula `FETCH FIRST`. A pesar de estas diferencias dialectales, el núcleo del lenguaje —sentencias como `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `CREATE TABLE`, entre otras— se comporta de forma suficientemente uniforme como para que un profesional con conocimiento sólido de un dialecto pueda adaptarse con relativa rapidez a otro.

---

2. Características Generales de SQL

SQL posee un conjunto de características que lo distinguen de otros lenguajes de programación o de acceso a datos. A continuación, se describen las más relevantes:

2.1 Lenguaje Declarativo

A diferencia de los lenguajes de programación imperativos (como C, Java o Python), en los que el programador especifica cómo debe ejecutarse cada operación paso a paso, SQL es un lenguaje declarativo: el programador especifica únicamente qué resultado desea obtener, y es el motor de base de datos —a través de su optimizador de consultas— quien decide el plan de ejecución más eficiente. Esta característica reduce significativamente la complejidad cognitiva del código y permite que mejoras en el motor de base de datos beneficien automáticamente las consultas existentes sin modificar el código fuente.

2.2 Independencia de Plataforma

SQL fue diseñado desde sus inicios con la intención de funcionar en diferentes plataformas de hardware y sistemas operativos. Un script SQL escrito para un entorno puede, en principio, ejecutarse en otro entorno compatible con el mismo estándar. Esta portabilidad es especialmente valiosa en entornos empresariales donde la infraestructura puede variar entre departamentos o donde existen políticas de migración de plataforma a largo plazo.

2.3 Estandarización Internacional

Como se mencionó en la introducción, SQL cuenta con un estándar internacional mantenido conjuntamente por ANSI e ISO. Este estándar proporciona un marco de referencia estable que garantiza la interoperabilidad mínima entre sistemas, facilita la portabilidad del conocimiento profesional, y sirve como base para la certificación técnica en bases de datos.

2.4 Capacidad de Manipulación de Datos Masiva

SQL permite operar sobre conjuntos completos de registros con una sola instrucción. Por ejemplo, una única sentencia `UPDATE` puede modificar millones de filas que cumplan cierto criterio, sin necesidad de iterar registro por registro mediante código procedimental. Esta orientación a conjuntos (set-oriented) es más eficiente que el procesamiento registro a registro y es fundamental para el rendimiento en bases de datos de gran escala.

2.5 Integración con Lenguajes de Programación

SQL puede integrarse con prácticamente cualquier lenguaje de programación moderno mediante diversas técnicas: SQL embebido (embedded SQL), interfaces de programación de aplicaciones como ODBC (Open Database Connectivity) y JDBC (Java Database Connectivity), ORMs (Object-Relational Mappers) como Hibernate, SQLAlchemy o Entity Framework, y librerías nativas de cliente. Esta capacidad de integración ha contribuido decisivamente a la adopción universal del lenguaje en el desarrollo de aplicaciones.

2.6 Soporte a Transacciones (ACID)

Los sistemas de bases de datos que implementan SQL cumplen generalmente con las propiedades ACID: Atomicidad (todas las operaciones de una transacción se realizan o ninguna), Consistencia (la base de datos siempre permanece en un estado válido), Aislamiento (las transacciones concurrentes no interfieren entre sí) y Durabilidad (los cambios confirmados persisten incluso ante fallos del sistema). SQL proporciona las instrucciones `BEGIN TRANSACTION`, `COMMIT` y `ROLLBACK` para gestionar transacciones de forma explícita, lo que lo hace adecuado para aplicaciones críticas como sistemas bancarios, de salud o de logística.

2.7 Seguridad y Control de Acceso

SQL incluye un sublenjuaje específico para la gestión de privilegios: el DCL (Data Control Language), cuyos comandos principales son `GRANT` (otorgar permisos) y `REVOKE` (revocar permisos). Mediante estos comandos es posible controlar con granularidad quién puede leer, insertar, modificar o eliminar datos en objetos específicos de la base de datos, lo que permite implementar modelos de seguridad basados en roles (RBAC) y el principio de mínimo privilegio.

2.8 Escalabilidad

Los motores de bases de datos que utilizan SQL están optimizados para escalar tanto verticalmente (mayor capacidad de hardware en un único servidor) como horizontalmente (distribución en clústeres de servidores). Funcionalidades como la partición de tablas, la replicación, los índices y las vistas materializadas son mecanismos que SQL y sus implementaciones ofrecen para mantener el rendimiento conforme los volúmenes de datos crecen. Sistemas como MySQL Cluster, PostgreSQL con Citus o Amazon Aurora demuestran la viabilidad de SQL a escala de petabytes.

2.9 Soporte a Consultas Complejas

SQL permite expresar consultas de gran complejidad mediante el uso de subconsultas, uniones (`JOIN`), funciones de agregación (`SUM`, `AVG`, `COUNT`, `MAX`, `MIN`), funciones de ventana (`OVER`, `PARTITION BY`, `ROW_NUMBER`), expresiones de tabla comunes (CTE, mediante `WITH`), y operaciones de conjunto (`UNION`, `INTERSECT`, `EXCEPT`). Esta riqueza expresiva hace de SQL una herramienta apropiada tanto para consultas simples de recuperación como para análisis estadístico avanzado.

2.10 Independencia de Datos

SQL separa la vista lógica de los datos (cómo los ve el usuario o la aplicación) de su almacenamiento físico (cómo los organiza el motor internamente). Esta independencia, contemplada en el modelo relacional de Codd, permite modificar la estructura de almacenamiento sin impactar las aplicaciones que acceden a los datos, y viceversa.

---

3. Lenguaje de Definición de Datos (LDD / DDL)

Definición Completa

El Lenguaje de Definición de Datos (LDD), denominado en inglés Data Definition Language (DDL), es el subconjunto de SQL dedicado a la definición, creación, modificación y eliminación de la estructura de los objetos que conforman una base de datos. Estos objetos incluyen, entre otros: bases de datos, tablas, columnas, índices, vistas, procedimientos almacenados, funciones, disparadores (triggers), secuencias y esquemas. En otras palabras, el DDL trabaja sobre el esquema de la base de datos —la descripción formal de su arquitectura— y no sobre los datos en sí mismos.

Propósito y Usos

El propósito fundamental del DDL es permitir a los administradores de bases de datos y a los desarrolladores diseñar y mantener la infraestructura sobre la cual residirán los datos. Algunas aplicaciones concretas incluyen:

- Modelado inicial: Crear la estructura de tablas, columnas y relaciones durante la fase de diseño de un sistema de información.
- Migraciones de esquema: Modificar la estructura existente para adaptarse a nuevos requisitos de negocio, sin perder los datos almacenados.
- Gestión del ciclo de vida: Eliminar objetos obsoletos o innecesarios para mantener la base de datos organizada y eficiente.
- Optimización: Crear índices para acelerar las consultas o particionar tablas para mejorar el rendimiento con grandes volúmenes de datos.

Cuándo se Utiliza DDL

El DDL se utiliza principalmente en las siguientes situaciones:

- Durante la fase de diseño e implementación de un nuevo sistema de base de datos.
- Al realizar una migración de versión de una aplicación que requiere cambios en el esquema.
- En los scripts de inicialización (setup scripts) que preparan el entorno de base de datos antes de ejecutar la aplicación.
- Cuando se necesita refactorizar el modelo de datos para mejorar el rendimiento o la coherencia.
- En entornos de pruebas y desarrollo, donde los esquemas se crean y destruyen frecuentemente.

Características que lo Distinguen

- Los comandos DDL generan un `COMMIT` implícito en la mayoría de los SGBDR, lo que significa que sus efectos son permanentes e inmediatos y no pueden deshacerse con `ROLLBACK` (con la excepción de algunos sistemas como PostgreSQL, que sí soporta DDL transaccional).
- Operan sobre metadatos: la información que el sistema guarda sobre su propia estructura (almacenada en el catálogo o diccionario de datos).
- Sus cambios son estructurales y persistentes, afectando el esquema físico y lógico de la base de datos.
- En MySQL 8.0, los comandos DDL son atómicos gracias al DDL atómico introducido en esta versión, que garantiza que un comando DDL que falla no deja la base de datos en un estado inconsistente.

---

4. Lenguaje de Manipulación de Datos (LMD / DML)

Definición Completa

El Lenguaje de Manipulación de Datos (LMD), denominado en inglés Data Manipulation Language (DML), es el subconjunto de SQL destinado a la inserción, modificación, eliminación y recuperación de los datos contenidos en las tablas de una base de datos. A diferencia del DDL, que trabaja sobre la estructura, el DML opera sobre el contenido —las filas o registros— de los objetos ya definidos. Los comandos fundamentales del DML son `INSERT`, `UPDATE`, `DELETE` y, en la clasificación práctica utilizada en muchos cursos de bases de datos, también `SELECT`.

Propósito y Usos

El DML es el sublenguaje de SQL con el que interactúan más intensamente las aplicaciones de software en su operación diaria. Sus usos principales son:

- Inserción de registros: Añadir nueva información a las tablas (altas de usuarios, registro de transacciones, carga de catálogos, etc.).
- Actualización de registros: Modificar información existente para reflejar cambios en la realidad del negocio (actualización de precios, cambio de dirección de un cliente, etc.).
- Eliminación de registros: Borrar información que ya no es relevante o que ha caducado.
- Consulta de datos: Recuperar información específica de una o más tablas para presentarla al usuario o procesarla en la capa de aplicación.

Cuándo se Utiliza DML

El DML se utiliza de forma continua durante toda la vida operativa de un sistema:

- En las operaciones CRUD (Create, Read, Update, Delete) que implementan la lógica de negocio de cualquier aplicación.
- En procesos de carga masiva de datos (batch processing) para importar grandes volúmenes de información.
- En informes y análisis de datos para extraer información estratégica.
- En procesos de limpieza de datos (data cleansing) para corregir inconsistencias.
- En la implementación de procedimientos almacenados y disparadores que automatizan operaciones complejas de negocio.

Características que lo Distinguen

- Los comandos DML son generalmente transaccionales: pueden agruparse dentro de una transacción y deshacerse con `ROLLBACK` si algo sale mal, o confirmarse de forma permanente con `COMMIT`.
- Operan sobre los datos (filas/registros), no sobre la estructura.
- Su impacto puede ser selectivo mediante el uso de la cláusula `WHERE`, que permite afectar solo a las filas que cumplen una condición específica.

- Son las instrucciones más frecuentemente ejecutadas en un sistema en producción, por lo que su optimización (mediante índices, estadísticas y planes de ejecución) es crítica para el rendimiento.

---

5. Diferencias entre LDD y LMD

La siguiente tabla comparativa resume las principales diferencias entre el Lenguaje de Definición de Datos y el Lenguaje de Manipulación de Datos:

Criterio		
LDD (DDL)		
LMD (DML)		
Propósito	Definir y gestionar la estructura de la base de datos	Manipular (insertar, actualizar, eliminar, consultar) los datos almacenados
Objetos que afecta	Esquema: tablas, índices, vistas, procedimientos, funciones, triggers	Registros (filas) dentro de las tablas existentes
¿Afecta datos?	No directamente (puede eliminarlos indirectamente con `DROP` o `TRUNCATE`)	Sí, directamente
¿Afecta estructura?	Sí, modifica el esquema de la base de datos	No, la estructura permanece inalterada
Transaccional	No en la mayoría de SGBDR (MySQL auto-commit implícito)	Sí, puede agruparse en transacciones explícitas
Reversible (ROLLBACK)	No (en MySQL; sí en PostgreSQL con DDL transaccional)	Sí, antes de hacer `COMMIT`
Ejemplos de comandos	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`	`INSERT`, `UPDATE`, `DELETE`, `SELECT`

---

6. Comandos DDL (LDD)

6.1 CREATE

6.1.1 Definición

`CREATE` es el comando DDL utilizado para crear nuevos objetos en la base de datos, tales como bases de datos, tablas, índices, vistas, procedimientos almacenados, funciones, disparadores y otros elementos del esquema. Es el punto de partida de cualquier diseño de base de datos.

6.1.2 Función

Su función principal es instanciar objetos nuevos en el catálogo del SGBDR, reservando los recursos necesarios (espacio en disco, estructuras de metadatos, etc.) y registrando la definición del objeto para su uso posterior. Sin `CREATE`, no existirían las estructuras sobre las cuales operar con DML.

6.1.3 Sintaxis Formal

```
-- Crear una base de datos
CREATE DATABASE [IF NOT EXISTS] nombre_base_datos
  [CHARACTER SET nombre_charset]
  [COLLATE nombre_collation];

-- Crear una tabla
CREATE TABLE [IF NOT EXISTS] nombre_tabla (
    nombre_columna1 tipo_dato [restricciones],
    nombre_columna2 tipo_dato [restricciones],
    ...
    [CONSTRAINT nombre_pk PRIMARY KEY (columna)],
    [CONSTRAINT nombre_fk FOREIGN KEY (columna) REFERENCES otra_tabla(columna)]
) [ENGINE=InnoDB] [DEFAULT CHARSET=utf8mb4];

-- Crear un índice
CREATE [UNIQUE] INDEX nombre_indice
    ON nombre_tabla (columna1 [ASC|DESC], ...);

-- Crear una vista
CREATE [OR REPLACE] VIEW nombre_vista AS
    SELECT ...;
```

6.1.4 Parámetros Principales

Parámetro		
Descripción		
Requerido		
`IF NOT EXISTS`	Evita error si el objeto ya existe; la operación se omite silenciosamente	No
`nombre_tabla`	Identificador único del objeto a crear dentro del esquema	Sí
`tipo_dato`	Define el dominio de valores de una columna (INT, VARCHAR, DATE, etc.)	Sí
`ENGINE`	Motor de almacenamiento a utilizar (InnoDB, MyISAM, MEMORY, etc.)	No (default: InnoDB)
`DEFAULT CHARSET`	Juego de caracteres predeterminado para la tabla	No
`COLLATE`	Orden de comparación de cadenas de texto	No
`PRIMARY KEY`	Define la columna o conjunto de columnas que identifican unívocamente cada fila	No (recomendado)
`FOREIGN KEY`	Establece una relación referencial con otra tabla	No

6.1.5 Restricciones

- El nombre del objeto no puede duplicar uno ya existente en el mismo esquema (a menos que se use `IF NOT EXISTS`).
- Los nombres de tabla y columna no pueden ser palabras reservadas de SQL sin usar comillas especiales.
- El número máximo de columnas por tabla en MySQL es de 1017 columnas.
- El tamaño máximo de una fila en InnoDB es de 65,535 bytes (con excepciones para tipos LOB).
- Los índices en columnas `BLOB` o `TEXT` requieren especificar una longitud de prefijo.

6.1.6 Opciones Avanzadas

- `CREATE TABLE ... LIKE`: Crea una nueva tabla con la misma estructura que una existente, sin copiar los datos.
- `CREATE TABLE ... AS SELECT`: Crea una nueva tabla y la rellena con los resultados de una consulta.
- `CREATE TEMPORARY TABLE`: Crea una tabla temporal que sólo existe durante la sesión actual.
- `CREATE TABLE` con `PARTITION BY`: Permite definir particionamiento horizontal para mejorar el rendimiento con grandes volúmenes de datos.

6.1.7 Ejemplo Práctico (MySQL 8.0)

```
-- Crear la base de datos del sistema universitario
CREATE DATABASE IF NOT EXISTS universidad_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

-- Seleccionar la base de datos
USE universidad_db;

-- Crear tabla de estudiantes
CREATE TABLE IF NOT EXISTS estudiantes (
    id_estudiante   INT             UNSIGNED AUTO_INCREMENT,
    matricula       VARCHAR(10)     NOT NULL UNIQUE,
    nombre          VARCHAR(60)     NOT NULL,
    apellido_p      VARCHAR(40)     NOT NULL,
    apellido_m      VARCHAR(40),
    fecha_nacimiento DATE           NOT NULL,
    correo          VARCHAR(100)    NOT NULL UNIQUE,
    id_carrera      TINYINT         UNSIGNED NOT NULL,
    fecha_registro  TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    activo          TINYINT(1)      NOT NULL DEFAULT 1,
    CONSTRAINT pk_estudiante PRIMARY KEY (id_estudiante)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

6.1.8 Explicación del Ejemplo

Se crea primero la base de datos `universidad_db` con soporte completo para UTF-8 (incluyendo emojis y caracteres de idiomas asiáticos). Luego se crea la tabla `estudiantes` con las siguientes características destacadas: la columna `id_estudiante` es una clave primaria numérica sin signo con autoincremento; `matricula` y `correo` tienen restricción `UNIQUE` para evitar duplicados; `fecha_registro` captura automáticamente la marca de tiempo de inserción; y `activo` funciona como bandera booleana para implementar eliminación lógica.

6.1.9 Resultado Esperado

La base de datos `universidad_db` queda creada en el servidor MySQL, y dentro de ella la tabla `estudiantes` aparece en el catálogo del sistema (`INFORMATION_SCHEMA.TABLES`). La tabla está vacía y lista para recibir registros mediante sentencias `INSERT`.

6.2 ALTER

6.2.1 Definición

`ALTER` es el comando DDL que permite modificar la estructura de un objeto existente en la base de datos sin necesidad de eliminarlo y recrearlo. Es fundamental para la evolución del esquema a lo largo del ciclo de vida del sistema.

6.2.2 Función

Permite realizar cambios estructurales como: agregar o eliminar columnas, cambiar el tipo de dato o las restricciones de una columna, renombrar columnas o tablas, agregar o eliminar claves primarias, foráneas o índices, y modificar propiedades del motor o conjunto de caracteres.

6.2.3 Sintaxis Formal

```
ALTER TABLE nombre_tabla
    ADD [COLUMN] nombre_col tipo_dato [restricciones] [FIRST | AFTER otra_col],
    DROP [COLUMN] nombre_col,
    MODIFY [COLUMN] nombre_col nuevo_tipo [restricciones],
    CHANGE [COLUMN] nombre_viejo nombre_nuevo tipo_dato [restricciones],
    RENAME [TO | AS] nuevo_nombre_tabla,
    ADD CONSTRAINT nombre_pk PRIMARY KEY (columna),
    DROP PRIMARY KEY,
    ADD CONSTRAINT nombre_fk FOREIGN KEY (col) REFERENCES otra_tabla(col),
    DROP FOREIGN KEY nombre_fk,
    ADD INDEX nombre_indice (columna),
    DROP INDEX nombre_indice,
    ENGINE = nuevo_motor,
    CONVERT TO CHARACTER SET nuevo_charset;
```

6.2.4 Parámetros Principales

Parámetro		
Descripción		
Requerido		
`ADD COLUMN`	Agrega una nueva columna a la tabla	Condicional
`DROP COLUMN`	Elimina una columna existente y todos sus datos	Condicional
`MODIFY COLUMN`	Cambia el tipo o restricciones de una columna existente	Condicional
`CHANGE COLUMN`	Renombra una columna y/o cambia su definición	Condicional
`RENAME TO`	Cambia el nombre de la tabla	Condicional
`FIRST / AFTER`	Especifica la posición de la nueva columna en la tabla	No

6.2.5 Restricciones

- Eliminar una columna que forma parte de un índice requiere eliminar primero el índice.
- Modificar una columna que contiene datos puede fallar si los datos existentes son incompatibles con el nuevo tipo.
- En tablas con millones de filas, `ALTER TABLE` puede ser una operación muy costosa en tiempo; existen herramientas como `pt-online-schema-change` para realizarlo sin bloqueo.
- No se puede eliminar una columna referenciada por una clave foránea de otra tabla.

6.2.6 Opciones Avanzadas

- `ALTER TABLE ... ALGORITHM=INPLACE`: Realiza el cambio sin reconstruir la tabla completa (cuando es posible).
- `ALTER TABLE ... LOCK=NONE`: Permite operaciones de lectura y escritura simultáneas durante el ALTER (online DDL).
- `ALTER DATABASE`: Modifica las propiedades de una base de datos (charset, collation).

6.2.7 Ejemplo Práctico (MySQL 8.0)

```
USE universidad_db;

-- Agregar columna de teléfono después de correo
ALTER TABLE estudiantes
    ADD COLUMN telefono VARCHAR(15) AFTER correo;

-- Modificar el tamaño del campo apellido_m
ALTER TABLE estudiantes
    MODIFY COLUMN apellido_m VARCHAR(50) DEFAULT 'N/A';

-- Cambiar el nombre y tipo de la columna activo
ALTER TABLE estudiantes
    CHANGE COLUMN activo estatus ENUM('activo','inactivo','egresado') NOT NULL DEFAULT 'activo';

-- Agregar un índice para agilizar búsquedas por apellido
ALTER TABLE estudiantes
    ADD INDEX idx_apellido_p (apellido_p);
```

6.2.8 Explicación del Ejemplo

La secuencia de `ALTER TABLE` ilustra las operaciones más comunes de evolución de esquema: primero se agrega la columna `telefono` en una posición específica; luego se amplía el tamaño de `apellido_m` y se le asigna un valor predeterminado; después se transforma la columna `activo` de un entero booleano a un tipo enumerado más expresivo con tres estados posibles; finalmente se crea un índice sobre `apellido_p` para optimizar las búsquedas por ese campo.

6.2.9 Resultado Esperado

La tabla `estudiantes` mantiene todos sus datos existentes pero su estructura se ha actualizado: ahora tiene la columna `telefono`, el campo `apellido_m` acepta hasta 50 caracteres con valor por defecto, el campo de estatus es un enumerado, y existe un índice auxiliar que acelera las consultas que filtren por apellido paterno.

6.3 DROP

6.3.1 Definición

`DROP` es el comando DDL que elimina de forma permanente un objeto de la base de datos, incluyendo su definición en el catálogo de metadatos y todos los datos que contenga (en el caso de tablas). Es la operación más destructiva del DDL.

6.3.2 Función

Permite eliminar bases de datos, tablas, índices, vistas, procedimientos almacenados, funciones, disparadores y otros objetos. Una vez ejecutado, el objeto y toda su información son irrecuperables a menos que exista una copia de seguridad.

6.3.3 Sintaxis Formal

```
-- Eliminar una base de datos
DROP DATABASE [IF EXISTS] nombre_base_datos;

-- Eliminar una tabla
DROP TABLE [IF EXISTS] nombre_tabla1 [, nombre_tabla2, ...];

-- Eliminar una vista
DROP VIEW [IF EXISTS] nombre_vista;

-- Eliminar un índice
DROP INDEX nombre_indice ON nombre_tabla;

-- Eliminar un procedimiento almacenado
DROP PROCEDURE [IF EXISTS] nombre_procedimiento;

-- Eliminar una función
DROP FUNCTION [IF EXISTS] nombre_funcion;
```

6.3.4 Parámetros Principales

Parámetro		
Descripción		
Requerido		
`IF EXISTS`	Suprime el error si el objeto no existe; la operación se ignora	No
`nombre_objeto`	Nombre del objeto a eliminar	Sí
`RESTRICT`	(Estándar SQL) Impide el DROP si el objeto tiene dependencias	No
`CASCADE`	(Estándar SQL) Elimina también los objetos dependientes automáticamente	No

6.3.5 Restricciones

- En MySQL, no se puede eliminar una tabla si otra tabla tiene una clave foránea que la referencia, a menos que se deshabilite temporalmente la verificación de claves foráneas (`SET FOREIGN_KEY_CHECKS = 0`).
- `DROP DATABASE` elimina todos los objetos contenidos en ella y el directorio de datos correspondiente.
- La operación es irreversible en MySQL; no existe `ROLLBACK` para `DROP TABLE`.
- Se requieren privilegios `DROP` sobre el objeto a eliminar.

6.3.6 Opciones Avanzadas

- En MySQL 8.0, `DROP TABLE` es atómica: si el servidor falla durante la operación, al reiniciar el sistema se encuentra en un estado consistente (la tabla existe o no existe, nunca a medias).
- Se pueden eliminar múltiples tablas en una sola sentencia separándolas por comas.

6.3.7 Ejemplo Práctico (MySQL 8.0)

```
USE universidad_db;

-- Crear tablas auxiliares de ejemplo para demostrar DROP
CREATE TABLE IF NOT EXISTS materias_obsoletas (
    id   INT PRIMARY KEY,
    nombre VARCHAR(100)
);

CREATE TABLE IF NOT EXISTS grupos_temporales (
    id   INT PRIMARY KEY,
    descripcion VARCHAR(200)
);

-- Eliminar una sola tabla
DROP TABLE IF EXISTS materias_obsoletas;

-- Eliminar múltiples tablas en una sola instrucción
DROP TABLE IF EXISTS grupos_temporales;

-- Crear y eliminar una vista
CREATE OR REPLACE VIEW vista_estudiantes_activos AS
    SELECT id_estudiante, nombre, apellido_p, correo
    FROM estudiantes
    WHERE estatus = 'activo';

DROP VIEW IF EXISTS vista_estudiantes_activos;
```

6.3.8 Explicación del Ejemplo

Se crean dos tablas auxiliares para demostrar su posterior eliminación. La cláusula `IF EXISTS` en ambos `DROP TABLE` garantiza que el script no genere un error si las tablas ya fueron eliminadas previamente, lo que es una buena práctica en scripts de migración o limpieza. También se muestra cómo crear y eliminar una vista. En todos los casos, los objetos desaparecen completamente del sistema.

6.3.9 Resultado Esperado

Las tablas `materias_obsoletas` y `grupos_temporales`, así como la vista `vista_estudiantes_activos`, son eliminadas permanentemente de la base de datos `universidad_db`. Si se ejecuta `SHOW TABLES`, estos objetos ya no aparecerán en el listado.

6.4 TRUNCATE

6.4.1 Definición

`TRUNCATE` es el comando DDL que elimina todos los registros de una tabla de forma rápida y eficiente, conservando la estructura (esquema) de la tabla intacta y lista para recibir nuevos datos. Se comporta conceptualmente como un `DELETE` sin cláusula `WHERE`, pero con importantes diferencias técnicas.

6.4.2 Función

Su función es vaciar completamente el contenido de una tabla de la manera más eficiente posible, generalmente borrando y recreando los archivos de datos internamente en lugar de eliminar fila por fila. Además, en la mayoría de los SGBDR, reinicia los contadores de autoincremento al valor inicial.

6.4.3 Sintaxis Formal

```
TRUNCATE [TABLE] nombre_tabla;
```

6.4.4 Parámetros Principales

Parámetro		
Descripción		
Requerido		
TABLE	Palabra clave opcional que precede al nombre de la tabla	No
`nombre_tabla`	Nombre de la tabla a vaciar	Sí

6.4.5 Restricciones

- No se puede usar `TRUNCATE` en una tabla que sea referenciada por una clave foránea activa de otra tabla (en MySQL con InnoDB y `FOREIGN_KEY_CHECKS = 1`).
- `TRUNCATE` no dispara los triggers de tipo `DELETE` definidos en la tabla.
- En MySQL, `TRUNCATE` genera un `COMMIT` implícito y no puede deshacerse con `ROLLBACK`.
- No admite cláusula `WHERE`, por lo que siempre afecta a todos los registros.

6.4.6 Opciones Avanzadas

- Para vaciar una tabla que tiene referencias de claves foráneas activas, se puede deshabilitar temporalmente la verificación: `SET FOREIGN_KEY_CHECKS = 0;` antes del `TRUNCATE` y `SET FOREIGN_KEY_CHECKS = 1;` después.
- En sistemas que soportan DDL transaccional (como PostgreSQL), `TRUNCATE` puede incluirse dentro de una transacción y deshacerse con `ROLLBACK`.

6.4.7 Ejemplo Práctico (MySQL 8.0)

```
USE universidad_db;

-- Crear tabla de log temporal para el ejemplo
CREATE TABLE IF NOT EXISTS log_accesos (
    id_log      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    id_usuario  INT UNSIGNED NOT NULL,
    fecha_hora  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    accion      VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

-- Insertar registros de ejemplo
INSERT INTO log_accesos (id_usuario, accion) VALUES
    (1, 'login'),
    (2, 'consulta_calificaciones'),
    (3, 'cambio_contrasena'),
    (1, 'logout');

-- Verificar contenido antes de truncar
SELECT COUNT(*) AS total_registros FROM log_accesos;
-- Resultado esperado: 4

-- Vaciar la tabla y reiniciar el autoincremento
TRUNCATE TABLE log_accesos;

-- Verificar contenido después de truncar
SELECT COUNT(*) AS total_registros FROM log_accesos;
-- Resultado esperado: 0

-- El próximo registro tendrá id_log = 1 (autoincremento reiniciado)
INSERT INTO log_accesos (id_usuario, accion) VALUES (5, 'login');
SELECT id_log, id_usuario, accion FROM log_accesos;
-- id_log será 1, no 5
```

6.4.8 Explicación del Ejemplo

Se crea una tabla de registro de accesos y se insertan cuatro registros. Antes del `TRUNCATE`, la tabla tiene 4 filas. Después de `TRUNCATE TABLE log_accesos`, la tabla queda completamente vacía y el contador de autoincremento se reinicia a 1. El siguiente registro insertado tendrá `id_log = 1`, lo que ilustra la diferencia con `DELETE`, que no reinicia el autoincremento.

6.4.9 Resultado Esperado

La tabla `log_accesos` queda vacía (0 registros). El contador de autoincremento se reinicia a 1. La estructura de la tabla (columnas, tipos, restricciones) permanece intacta.

---

7. Comandos DML (LMD)

7.1 INSERT

7.1.1 Definición

`INSERT` es el comando DML utilizado para agregar nuevas filas (registros) a una tabla existente en la base de datos. Es la operación fundamental de creación de datos dentro del ciclo CRUD.

7.1.2 Función

Permite añadir uno o múltiples registros a una tabla especificando los valores para cada columna. Los valores deben ser compatibles con los tipos de dato y restricciones definidas en el esquema de la tabla.

7.1.3 Sintaxis Formal

```
-- Insertar un registro especificando columnas
INSERT INTO nombre_tabla (col1, col2, col3, ...)
    VALUES (val1, val2, val3, ...);

-- Insertar múltiples registros
INSERT INTO nombre_tabla (col1, col2, col3)
    VALUES
        (val1a, val2a, val3a),
        (val1b, val2b, val3b),
        (val1c, val2c, val3c);

-- Insertar resultado de un SELECT
INSERT INTO nombre_tabla (col1, col2)
    SELECT col_origen1, col_origen2
    FROM tabla_origen
    WHERE condicion;

-- Insertar o actualizar si ya existe (MySQL)
INSERT INTO nombre_tabla (col1, col2)
    VALUES (val1, val2)
    ON DUPLICATE KEY UPDATE col2 = VALUES(col2);
```

7.1.4 Parámetros Principales

Parámetro		
Descripción		
Requerido		
`nombre_tabla`	Tabla destino donde se insertarán los datos	Sí
`(col1, col2, ...)`	Lista de columnas a poblar; si se omite, se esperan valores para todas	No (recomendado)
`VALUES (...)`	Lista de valores correspondientes a las columnas	Sí
`ON DUPLICATE KEY UPDATE`	Especifica qué hacer si se viola una restricción UNIQUE o PRIMARY KEY	No

7.1.5 Restricciones

- Los valores deben cumplir con los tipos de dato, longitudes y restricciones (`NOT NULL`, `UNIQUE`, `CHECK`, `FOREIGN KEY`) definidas en la tabla.
- Si no se especifica la lista de columnas, deben proporcionarse valores para todas las columnas en el orden en que fueron definidas.
- Las columnas `AUTO_INCREMENT` y las que tienen `DEFAULT` pueden omitirse de la lista.
- Insertar un valor duplicado en una columna con restricción `UNIQUE` o `PRIMARY KEY` genera un error, a menos que se use `ON DUPLICATE KEY UPDATE` o `INSERT IGNORE`.

7.1.6 Opciones Avanzadas

- `INSERT IGNORE`: Omite silenciosamente los errores de duplicado o de restricción.
- `INSERT ... SELECT`: Permite copiar datos entre tablas o insertar datos derivados de consultas complejas.
- `INSERT ... ON DUPLICATE KEY UPDATE`: Implementa la operación "upsert" (insertar o actualizar si ya existe).
- `REPLACE INTO`: Elimina la fila existente e inserta la nueva si se detecta un duplicado (equivalente a DELETE + INSERT).

7.1.7 Ejemplo Práctico (MySQL 8.0)

```
USE universidad_db;

-- Insertar un único estudiante
INSERT INTO estudiantes (matricula, nombre, apellido_p, apellido_m, fecha_nacimiento, correo, id_carrera)
    VALUES ('2026001', 'Ana Sofía', 'Martínez', 'López', '2004-03-15', 'ana.martinez@universidad.edu', 1);

-- Insertar múltiples estudiantes en una sola instrucción
INSERT INTO estudiantes (matricula, nombre, apellido_p, apellido_m, fecha_nacimiento, correo, id_carrera)
    VALUES
        ('2026002', 'Carlos Eduardo', 'Ramírez', 'Torres', '2003-07-22', 'carlos.ramirez@universidad.edu', 2),
        ('2026003', 'Lucía', 'Hernández', 'García', '2004-11-08', 'lucia.hernandez@universidad.edu', 1),
        ('2026004', 'Miguel Ángel', 'Sánchez', NULL, '2003-01-30', 'miguel.sanchez@universidad.edu', 3);
```

7.1.8 Explicación del Ejemplo

El primer `INSERT` agrega un único registro de estudiante especificando explícitamente las columnas a poblar. Se omiten `id_estudiante` (AUTO_INCREMENT), `telefono` (permite NULL), `fecha_registro` (DEFAULT CURRENT_TIMESTAMP) y `estatus` (DEFAULT 'activo'). El segundo `INSERT` utiliza la sintaxis de valores múltiples para agregar tres estudiantes en una sola operación, lo que es más eficiente que tres sentencias `INSERT` separadas. El cuarto estudiante tiene `apellido_m = NULL`, lo que es válido porque esa columna no tiene restricción `NOT NULL`.

7.1.9 Resultado Esperado

La tabla `estudiantes` contiene ahora 4 registros, con `id_estudiante` asignados automáticamente (1, 2, 3, 4), `fecha_registro` con la marca de tiempo actual y `estatus = 'activo'` para todos.

7.2 UPDATE

7.2.1 Definición

`UPDATE` es el comando DML que permite modificar los valores de una o más columnas en registros existentes de una tabla. Es la operación de actualización dentro del ciclo CRUD.

7.2.2 Función

Permite cambiar el contenido de filas que cumplan una condición especificada en la cláusula `WHERE`. Si se omite `WHERE`, todos los registros de la tabla son afectados, lo que suele ser un error no intencionado de consecuencias graves.

7.2.3 Sintaxis Formal

```
-- Actualizar con condición (uso recomendado)
UPDATE nombre_tabla
    SET col1 = nuevo_val1,
        col2 = nuevo_val2,
        ...
    WHERE condicion;

-- Actualizar con JOIN (actualizar basado en otra tabla)
UPDATE tabla1
    INNER JOIN tabla2 ON tabla1.id = tabla2.id_fk
    SET tabla1.columna = tabla2.valor_origen
    WHERE tabla2.condicion = valor;

-- Actualizar usando ORDER BY y LIMIT
UPDATE nombre_tabla
    SET columna = valor
    WHERE condicion
    ORDER BY columna_orden
    LIMIT n;
```

7.2.4 Parámetros Principales

Parámetro		
Descripción		
Requerido		

Parámetro		
`nombre_tabla`	Tabla cuyos registros serán actualizados	Sí
`SET`	Lista de asignaciones columna = valor	Sí
`WHERE`	Condición que filtra qué registros serán afectados	No (omitirlo actualiza TODOS)
`ORDER BY`	Define el orden en que se actualizan las filas	No
`LIMIT`	Restringe el número de filas a actualizar	No

7.2.5 Restricciones

- Los nuevos valores deben ser compatibles con el tipo de dato y las restricciones de cada columna.
- No se puede actualizar la misma tabla que se usa en una subconsulta `FROM` directamente (en MySQL).
- La omisión accidental de `WHERE` es uno de los errores más costosos en bases de datos en producción.
- Actualizar una columna de clave primaria puede violar restricciones de integridad referencial.

7.2.6 Opciones Avanzadas

- Se pueden utilizar expresiones en el lado derecho de la asignación, incluyendo funciones y operaciones aritméticas: `SET saldo = saldo * 1.05`.
- MySQL 8.0 soporta CTEs (`WITH`) en conjunto con `UPDATE` para lógica más compleja.
- `UPDATE ... IGNORE` omite errores de restricciones y continúa con las siguientes filas.

7.2.7 Ejemplo Práctico (MySQL 8.0)

```
USE universidad_db;

-- Actualizar el teléfono de un estudiante específico
UPDATE estudiantes
    SET telefono = '555-123-4567'
    WHERE matricula = '2026001';

-- Actualizar el estatus de varios estudiantes a 'egresado'
UPDATE estudiantes
    SET estatus = 'egresado',
        telefono = COALESCE(telefono, 'No registrado')
    WHERE id_carrera = 1
      AND YEAR(fecha_nacimiento) &lt;= 2003;

-- Actualizar masivamente con expresión
UPDATE estudiantes
    SET correo = CONCAT(LOWER(nombre), '.', LOWER(apellido_p), '@nueva.universidad.edu')
    WHERE estatus = 'activo';
```

7.2.8 Explicación del Ejemplo

El primer `UPDATE` modifica únicamente el teléfono de la estudiante con matrícula `2026001`. El segundo `UPDATE` cambia el estatus a 'egresado' para todos los estudiantes de la carrera 1 nacidos en 2003 o antes, y además asigna el texto 'No registrado' si no tienen teléfono (usando `COALESCE`). El tercer `UPDATE` actualiza masivamente el correo de todos los estudiantes activos usando una expresión que construye la dirección a partir de su nombre y apellido.

7.2.9 Resultado Esperado

Los registros de estudiantes afectados por las condiciones `WHERE` reflejan los nuevos valores. Los registros que no cumplieron las condiciones permanecen sin cambios.

7.3 DELETE

7.3.1 Definición

`DELETE` es el comando DML que elimina filas de una tabla que cumplan la condición especificada en la cláusula `WHERE`. A diferencia de `TRUNCATE`, `DELETE` es transaccional y opera fila por fila, lo que activa los triggers correspondientes.

7.3.2 Función

Permite borrar selectivamente registros de una tabla de forma controlada y reversible (dentro de una transacción). Si se omite la cláusula `WHERE`, elimina todos los registros de la tabla.

7.3.3 Sintaxis Formal

```
-- Eliminar filas con condición
DELETE FROM nombre_tabla
    WHERE condicion;

-- Eliminar filas con JOIN
DELETE tabla1
    FROM tabla1
    INNER JOIN tabla2 ON tabla1.id = tabla2.id_fk
    WHERE tabla2.condicion = valor;

-- Eliminar con ORDER BY y LIMIT
DELETE FROM nombre_tabla
    WHERE condicion
    ORDER BY columna
    LIMIT n;
```

7.3.4 Parámetros Principales

Parámetro		
Descripción		
Requerido		
`FROM`	Especifica la tabla origen de la eliminación	Sí
`WHERE`	Condición que determina qué filas serán eliminadas	No (omitirlo borra TODAS las filas)
`ORDER BY`	Define el orden de eliminación (útil con `LIMIT`)	No
`LIMIT`	Restringe el número de filas a eliminar	No

7.3.5 Restricciones

- No se pueden eliminar filas que son referenciadas por claves foráneas de otras tablas (si las restricciones están activas).
- `DELETE` dispara los triggers `BEFORE DELETE` y `AFTER DELETE`.
- Es más lento que `TRUNCATE` para borrar todos los registros, ya que opera fila a fila.
- Dentro de una transacción, puede deshacerse con `ROLLBACK`.

7.3.6 Opciones Avanzadas

- `DELETE IGNORE`: Continúa la operación aunque se produzcan errores en algunas filas.
- Uso de subconsultas en la cláusula `WHERE` para identificar filas a eliminar de forma dinámica.
- Uso del patrón de eliminación lógica (soft delete): en lugar de `DELETE`, se actualiza una columna `activo = 0` o `estatus = 'inactivo'`, preservando el historial.

7.3.7 Ejemplo Práctico (MySQL 8.0)

```
USE universidad_db;

-- Eliminar un estudiante específico por matrícula
DELETE FROM estudiantes
    WHERE matricula = '2026004';

-- Eliminar estudiantes inactivos cuya matrícula sea de años anteriores
-- (simulación de purga anual de registros antiguos)
DELETE FROM estudiantes
    WHERE estatus = 'inactivo'
      AND YEAR(fecha_registro) &lt; YEAR(CURDATE()) - 2;

-- Uso dentro de una transacción (patrón seguro)
START TRANSACTION;

DELETE FROM log_accesos
    WHERE fecha_hora &lt; DATE_SUB(NOW(), INTERVAL 30 DAY);

-- Verificar cuántas filas se eliminaron
-- Si es correcto: COMMIT; Si no: ROLLBACK;
COMMIT;
```

7.3.8 Explicación del Ejemplo

El primer `DELETE` elimina el estudiante con matrícula `2026004` de forma precisa. El segundo `DELETE` purga registros de estudiantes inactivos que llevan más de dos años registrados en el sistema, usando funciones de fecha para determinar la antigüedad. El tercer ejemplo muestra el patrón seguro de uso de `DELETE` dentro de una transacción explícita: si el resultado es el esperado, se hace `COMMIT`; si algo está mal, se puede hacer `ROLLBACK` para revertir la eliminación.

7.3.9 Resultado Esperado

Los registros que cumplen las condiciones `WHERE` son eliminados permanentemente (si se confirma con `COMMIT`). Los registros que no cumplen las condiciones permanecen intactos. El número de filas afectadas puede verificarse con `ROW_COUNT()`.

7.4 SELECT

7.4.1 Definición

`SELECT` es el comando DML de mayor importancia y complejidad en SQL, utilizado para consultar y recuperar datos almacenados en una o más tablas. Genera un conjunto de resultados (result set) que puede proyectarse, filtrarse, ordenarse y agrupase según las necesidades del usuario o la aplicación.

7.4.2 Función

Su función es recuperar datos de la base de datos aplicando proyecciones (selección de columnas), restricciones (filtros con `WHERE`), combinaciones de tablas (`JOIN`), agrupaciones (`GROUP BY`), ordenamientos (`ORDER BY`) y límites de resultados (`LIMIT`), entre muchas otras operaciones.

7.4.3 Sintaxis Formal

```
SELECT [DISTINCT] col1 [AS alias1], col2 [AS alias2], expresion, ...
    FROM tabla_principal [alias_tabla]
    [JOIN otra_tabla ON condicion_join]
    [WHERE condicion_filtro]
    [GROUP BY col_agrupacion]
    [HAVING condicion_sobre_grupos]
    [ORDER BY col_orden [ASC | DESC]]
    [LIMIT numero_filas [OFFSET inicio]];
```

7.4.4 Parámetros Principales

Parámetro		
Descripción		
Requerido		

Parámetro		
`DISTINCT`	Elimina filas duplicadas del resultado	No
`FROM`	Especifica la(s) tabla(s) de origen de los datos	Sí (generalmente)
`WHERE`	Filtra las filas antes de agrupación o proyección	No
`GROUP BY`	Agrupa las filas por el valor de una o más columnas	No
`HAVING`	Filtra grupos después de la agrupación (similar a `WHERE` para grupos)	No
`ORDER BY`	Define el criterio y dirección de ordenamiento del resultado	No
`LIMIT`	Restringe el número de filas retornadas	No
`OFFSET`	Define desde qué fila comenzar (útil para paginación)	No

7.4.5 Restricciones

- Se deben tener privilegios `SELECT` sobre los objetos consultados.
- Las columnas referenciadas en `GROUP BY` deben aparecer en la lista `SELECT` o ser funciones de agregación.
- En MySQL con el modo `ONLY_FULL_GROUP_BY` activado (default en MySQL 8.0), todas las columnas no agregadas en `SELECT` deben estar en `GROUP BY`.
- Consultas mal optimizadas sobre tablas grandes sin índices pueden tener impacto severo en el rendimiento.

7.4.6 Opciones Avanzadas

- Funciones de ventana (`OVER`, `PARTITION BY`, `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LAG`, `LEAD`): análisis avanzado de conjuntos de datos.
- CTE - Common Table Expressions (`WITH cte_name AS (...)`): subconsultas nombradas y reutilizables.
- Subconsultas correlacionadas: consultas anidadas que hacen referencia a la consulta externa.
- Tipos de JOIN: `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`, `CROSS JOIN`, `SELF JOIN`.
- Operadores de conjunto: `UNION`, `UNION ALL`, `INTERSECT`, `EXCEPT`.

7.4.7 Ejemplo Práctico (MySQL 8.0)

```
USE universidad_db;

-- Consulta básica: todos los estudiantes activos ordenados por apellido
SELECT
    id_estudiante,
    CONCAT(nombre, ' ', apellido_p, ' ', COALESCE(apellido_m, '')) AS nombre_completo,
    matricula,
    correo,
    fecha_registro
FROM estudiantes
WHERE estatus = 'activo'
ORDER BY apellido_p ASC, nombre ASC;

-- Consulta con agrupación: conteo de estudiantes por carrera
SELECT
    id_carrera,
    COUNT(*) AS total_estudiantes,
    MIN(fecha_nacimiento) AS estudiante_mayor,
    MAX(fecha_nacimiento) AS estudiante_menor
FROM estudiantes
WHERE estatus = 'activo'
GROUP BY id_carrera
HAVING COUNT(*) &gt; 0
ORDER BY total_estudiantes DESC;

-- Consulta con paginación (página 1, 10 registros por página)
SELECT
    id_estudiante,
    nombre,
    apellido_p,
    matricula
FROM estudiantes
ORDER BY id_estudiante
LIMIT 10 OFFSET 0;
```

7.4.8 Explicación del Ejemplo

La primera consulta recupera todos los estudiantes activos, construye su nombre completo concatenando nombre y apellidos (con `COALESCE` para manejar el apellido materno nulo), y ordena el resultado alfabéticamente. La segunda consulta agrupa a los estudiantes por carrera y calcula estadísticas: total de estudiantes, fecha de nacimiento más antigua y más reciente dentro de cada carrera, filtrando grupos con `HAVING`. La tercera consulta implementa paginación usando `LIMIT` y `OFFSET`, un patrón fundamental en aplicaciones web que muestran listados por páginas.

7.4.9 Resultado Esperado

La primera consulta retorna una tabla con los estudiantes activos ordenados alfabéticamente. La segunda retorna una fila por cada carrera con sus estadísticas. La tercera retorna hasta 10 estudiantes de la primera página, ordenados por ID.

---

8. Tabla Taxonómica de Comandos SQL

La siguiente tabla clasifica los ocho comandos estudiados según múltiples criterios técnicos relevantes:

Categoría						
Comando						
Función Principal						
Afecta Estructura						
Afecta Datos						
Reversible						
Nivel de Riesgo						

Categoría						
DDL (LDD)	`CREATE`	Crear nuevos objetos en el esquema	□ Sí	□ No	□ No	□ Medio
DDL (LDD)	`ALTER`	Modificar la estructura de objetos existentes	□ Sí	⚠ Indirectamente	□ No	□ Medio-Alto
DDL (LDD)	`DROP`	Eliminar objetos del esquema con sus datos	□ Sí	□ Sí (destruye)	□ No	□ Muy Alto
DDL (LDD)	`TRUNCATE`	Vaciar todos los registros de una tabla	□ No	□ Sí (destruye)	□ No	□ Alto
DML (LMD)	`INSERT`	Agregar nuevas filas a una tabla	□ No	□ Sí (añade)	□ Sí (en tx)	□ Bajo-Medio
DML (LMD)	`UPDATE`	Modificar valores en filas existentes	□ No	□ Sí (modifica)	□ Sí (en tx)	□ Medio-Alto
DML (LMD)	`DELETE`	Eliminar filas específicas de una tabla	□ No	□ Sí (elimina)	□ Sí (en tx)	□ Alto
DML (LMD)	`SELECT`	Consultar y recuperar datos sin modificarlos	□ No	□ No (solo lee)	N/A (solo lectura)	□ Bajo

Leyenda de Nivel de Riesgo:

- □ Bajo: Operación de solo lectura o de impacto mínimo y reversible.
- □ Medio / Medio-Alto: Puede afectar datos o estructura; requiere cuidado y validación previa.
- □ Alto / Muy Alto: Operación destructiva e irreversible; requiere respaldo previo y autorización.

---

9. Diferencias entre DDL, DML y DQL

¿Qué es DQL (Data Query Language)?

El Lenguaje de Consulta de Datos (Data Query Language - DQL) es una categoría teórica dentro de la clasificación de los sublengajes de SQL que agrupa específicamente las instrucciones destinadas a recuperar información de la base de datos sin modificarla. En la mayoría de las clasificaciones académicas estrictas, el DQL está compuesto únicamente por el comando `SELECT` (junto con sus cláusulas subordinadas: `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY`, `LIMIT`). Esta distinción enfatiza la naturaleza fundamentalmente diferente de la operación de consulta con respecto a las operaciones de modificación de datos.

¿Por Qué SELECT se Clasifica como DQL en la Teoría?

Desde una perspectiva teórica, `SELECT` merece una categoría propia porque:

1. No produce efectos secundarios: A diferencia de `INSERT`, `UPDATE` y `DELETE`, `SELECT` no altera el estado de la base de datos; es una operación de solo lectura (read-only).
2. No requiere transacciones de escritura: Las consultas `SELECT` no necesitan ser incluidas en transacciones para garantizar consistencia, aunque pueden ser parte de ellas para obtener lecturas consistentes.
3. Principio de responsabilidad única: Separar la lectura de la escritura clarifica el diseño del código y facilita la aplicación de patrones como CQRS (Command Query Responsibility Segregation).
4. Privilegios diferenciados: En la gestión de seguridad, el privilegio `SELECT` se otorga de forma independiente a los privilegios de escritura (`INSERT`, `UPDATE`, `DELETE`), lo que refuerza su naturaleza distinta.

¿Por Qué en Esta Actividad Académica se Incluye SELECT dentro de LMD?

En numerosos libros de texto, cursos universitarios y certificaciones de nivel introductorio, SELECT se agrupa dentro del DML por las siguientes razones pragmáticas:

- Simplicidad pedagógica: Para efectos de enseñanza, resulta más accesible clasificar todo lo que "hace algo con los datos" (ya sea leerlos o modificarlos) como DML, reservando el DDL para las operaciones de estructura.
- Tradición histórica: Las primeras versiones del estándar SQL no distinguían formalmente entre DML y DQL; ambos se consideraban parte del mismo grupo de sentencias que operan sobre los datos (en contraposición a las que definen la estructura).
- Practicidad en el desarrollo: Desde la perspectiva de un desarrollador de aplicaciones, SELECT y las demás instrucciones DML comparten el mismo contexto de trabajo: operar dentro de la capa de acceso a datos, interactuar con registros existentes y ser parte de la lógica transaccional del sistema.

Tabla Comparativa DDL vs DML vs DQL

Criterio			
DDL			
DML			
DQL			
Significado	Data Definition Language	Data Manipulation Language	Data Query Language
Traducción	Lenguaje de Definición de Datos	Lenguaje de Manipulación de Datos	Lenguaje de Consulta de Datos
Objeto de operación	Estructura (esquema)	Registros (filas/columnas de datos)	Registros (solo lectura)
Modifica datos	No (indirectamente con DROP)	Sí	No
Modifica estructura	Sí	No	No
Transaccional	No (en MySQL)	Sí	N/A
Reversible	No (en MySQL)	Sí (con ROLLBACK)	N/A
Comandos principales	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`	`INSERT`, `UPDATE`, `DELETE`	`SELECT`
Requiere privilegio	`CREATE`, `ALTER`, `DROP`	`INSERT`, `UPDATE`, `DELETE`	`SELECT`
Afecta al catálogo	Sí	No	No

---

10. Conclusiones

1. SQL es el fundamento universal del acceso a datos relacionales. A lo largo de más de cinco décadas de evolución, el lenguaje SQL ha demostrado una resiliencia y adaptabilidad notables. Desde su origen en los laboratorios de IBM hasta convertirse en estándar ANSI/ISO, SQL ha transformado la forma en que los sistemas de información almacenan, recuperan y gestionan datos. Su carácter declarativo, su independencia de plataforma y su estandarización internacional lo posicionan como una competencia técnica indispensable para cualquier profesional del área de tecnologías de la información en el siglo XXI.
2. La distinción entre DDL y DML refleja dos niveles fundamentales de abstracción en las bases de datos relacionales. El DDL opera sobre el esquema —la arquitectura conceptual y física de la información—, mientras que el DML opera sobre el contenido —los datos concretos almacenados en dicha arquitectura—. Comprender esta separación es esencial para el diseño correcto de sistemas de bases de datos: el esquema debe ser estable y bien planificado (DDL), mientras que los datos fluyen dinámicamente durante toda la vida del sistema (DML). Confundir ambos niveles lleva a diseños frágiles, difíciles de mantener y propensos

a errores graves.

3. El uso responsable de los comandos destructivos (DROP, TRUNCATE, DELETE sin WHERE) requiere disciplina profesional y salvaguardas técnicas. Los comandos más peligrosos de SQL —`DROP`, `TRUNCATE` y `DELETE` sin cláusula `WHERE`— pueden destruir grandes volúmenes de información en fracciones de segundo de forma irreversible. Las mejores prácticas de la industria establecen controles como: revisión de código por pares (code review) para cambios de esquema, respaldos automáticos antes de ejecutar DDL en producción, uso de transacciones explícitas para operaciones DML críticas, y principio de mínimo privilegio para limitar quién puede ejecutar estas instrucciones.

4. El dominio de SELECT es la habilidad de mayor impacto práctico en el trabajo cotidiano con bases de datos. Si bien todos los comandos SQL son necesarios, `SELECT` es, con diferencia, el más frecuentemente utilizado en la práctica profesional. Las consultas de recuperación de datos son la base del análisis de negocio, la generación de reportes, la depuración de sistemas y el desarrollo de interfaces de usuario. La capacidad de escribir consultas eficientes —aprovechando `JOIN`s, funciones de agregación, CTEs y funciones de ventana— distingue a un desarrollador competente de uno sobresaliente. Un `SELECT` mal escrito puede ser la causa raíz de problemas de rendimiento que afecten a miles de usuarios simultáneamente.

5. La clasificación de SELECT como DQL versus DML ilustra la importancia de contextualizar el conocimiento técnico. La discusión sobre si `SELECT` pertenece al DML o al DQL no es un debate trivial: refleja diferentes niveles de rigor teórico y diferentes necesidades pedagógicas. Académicamente, la distinción DQL subraya el principio de separación entre operaciones de lectura y escritura, que tiene implicaciones arquitectónicas importantes (como el patrón CQRS en sistemas distribuidos). Pragmáticamente, agrupar `SELECT` con el DML simplifica el aprendizaje introductorio. Un profesional maduro comprende ambas perspectivas y sabe aplicar la clasificación apropiada según el contexto: rigor teórico en el diseño de arquitecturas, pragmatismo en la implementación diaria.

Bibliografía

Chamberlin, D. D., & Boyce, R. F. (1974). SEQUEL: A structured English query language. Proceedings of the 1974 ACM SIGFIDET Workshop on Data Description, Access and Control, 249–264. https://doi.org/10.1145/800296.811515

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387. https://doi.org/10.1145/362384.362685

Date, C. J. (2004). An introduction to database systems (8th ed.). Addison-Wesley.

Elmasri, R., & Navathe, S. B. (2015). Fundamentals of database systems (7th ed.). Pearson Education.

Oracle Corporation. (2024). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

Ramakrishnan, R., & Gehrke, J. (2003). Database management systems (3rd ed.). McGraw-Hill.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database system concepts (7th ed.). McGraw-Hill Education.

Connolly, T., & Begg, C. (2015). Database systems: A practical approach to design, implementation, and management (6th ed.). Pearson Education.